TenXEngage

Redemption Store × XTRM API Integration Mapping Document

Full entity, service, and API-level mapping of every XTRM XAPI call required by the tenXengage Redemption Store feature — with request/response schemas, field-by-field transformations, data gaps, and open questions.

Vendor: XTRM (Cash Payouts)
Version: 1.0 — April 2026
Classification: Highly Confidential
Sandbox Base URL: https://xapisandbox.xtrm.com/API/V4/
Production Base URL: https://xapi.xtrm.com/API/V4/
Auth: OAuth 2.0 Bearer Token

■ *IMPORTANT: The prompt engineering document references POST /v1/rewards as the XTRM payout endpoint. The actual XTRM XAPI uses POST /API/V4/Fund/TransferFund with a completely different request schema. All mappings in this document use the real XTRM XAPI endpoints. Resolve this discrepancy with XTRM before coding XtrmAdapter.*

PART 1 — DATA MODEL

Entity: RewardWallet

Tenant-scoped (TenantAware + clientId + @Filter). The availableBalance and reservedBalance fields change as a direct result of XTRM payout webhook events. No XTRM API reads from this entity — it is purely internal state.

`ownerType`	Enum: INDIVIDUAL | COMPANY
`userId`	UUID (nullable FK → users)
`partnerCompanyId`	UUID (nullable FK → partner_companies)
`currencyType`	Enum: CASH | POINTS | CREDITS | TICKETS
`availableBalance`	BigDecimal NUMERIC(19,4) — updated by XTRM webhook
`reservedBalance`	BigDecimal NUMERIC(19,4) — updated by XTRM webhook
`totalEarned`	BigDecimal NUMERIC(19,4)
`[inherited]`	id UUID, clientId UUID, createdAt Instant, updatedAt Instant

Entity: LedgerEntry

Tenant-scoped. Immutable after creation — no @PreUpdate. Written by WalletService in response to XTRM webhook events: SUCCEEDED → entryType=DEBIT / sourceType=REDEMPTION_DEBIT; FAILED/CANCELLED → entryType=RELEASE / sourceType=REDEMPTION_RELEASE.

`walletId`	UUID (FK → reward_wallet)
`entryType`	Enum: CREDIT | DEBIT | RESERVE | RELEASE | RETURN_CREDIT | EXPIRY_DEBIT
`sourceType`	Enum: REWARD_EARNED | REDEMPTION_RESERVE | REDEMPTION_DEBIT | REDEMPTION_RELEASE | RETURN_CREDIT | ADJUSTMENT | CLAWBACK
`currencyType`	CurrencyType enum
`amount`	BigDecimal (always positive, CHECK > 0)
`relatedTransactionId`	UUID (nullable — links to RedemptionTransaction)
`[inherited]`	id UUID, clientId UUID, createdAt Instant, updatedAt Instant

Entity: RedemptionCatalogItem

Platform-level (NOT tenant-scoped — no clientId, no @Filter). Items with category=CASH route to XTRM. The providerItemId is NOT used for CASH items because XTRM is a payout service, not an item catalog.

`name`	String
`category`	Enum: CASH | NON_CASH — CASH items route to XTRM
`subCategory`	String
`currencyType`	CurrencyType

minimumAmount	BigDecimal
defaultProcessingMode	Enum: INSTANT | BATCH | APPROVAL_REQUIRED
providerItemId	String — NOT USED for CASH/XTRM items (Xoxoday only)
geographicScope	String[] (TEXT[])
isReturnable	Boolean — always false for CASH items
isActive	Boolean
[inherited]	id UUID, createdAt Instant, updatedAt Instant (no clientId)

Entity: ClientRedemptionConfig

Tenant-scoped. processingModeOverride determines whether XTRM is called immediately (INSTANT), scheduled (BATCH), or held for approval. minimumTransactionAmount gates the minimum amount sent to XTRM.

catalogItemId	UUID (FK → redemption_catalog_item)
isEnabled	Boolean — must be true for XTRM to be called
processingModeOverride	Enum (nullable) — overrides catalog item default
minimumWalletBalance	BigDecimal — wallet must exceed this before redemption
minimumTransactionAmount	BigDecimal — XTRM payout must be at least this amount
returnWindowDays	Integer — N/A for CASH (returns blocked)
[inherited]	id UUID, clientId UUID, createdAt Instant, updatedAt Instant

Entity: RedemptionTransaction

Tenant-scoped. Central XTRM entity. id is passed to XTRM as IssuerTransactionId. XTRM's TransactionId is stored back in providerTransactionId. Status lifecycle is driven by XTRM webhooks.

walletId	UUID (FK → reward_wallet)
catalogItemId	UUID (FK → redemption_catalog_item)
requestedBy	UUID (FK → users) — used to resolve recipient for XTRM
amount	BigDecimal — sent as PaymentAmount to XTRM
currencyType	CurrencyType — mapped to ISO 4217 for XTRM
status	TransactionStatus (reused platform enum)
processingMode	Enum: INSTANT | BATCH | APPROVAL_REQUIRED
providerName	Enum: XTRM | XOXODAY
providerTransactionId	String (nullable) — XTRM's TransactionId stored here
approvedBy	UUID (nullable)
approvedAt	Instant (nullable)
completedAt	Instant (nullable) — set on XTRM SUCCEEDED webhook
failureReason	String (nullable) — set on XTRM FAILED webhook
returnId	UUID (nullable) — N/A for CASH/XTRM
[inherited]	id UUID, clientId UUID, createdAt Instant, updatedAt Instant

Entity: RedemptionReturn

✕ *No XTRM involvement. Cash (XTRM) redemptions are explicitly non-returnable. ReturnsService.submitReturnRequest() throws BusinessRuleException if catalogItem.category = CASH. No XTRM return endpoint exists or is needed.*

`redemptionTransactionId`	UUID (unique FK → redemption_transaction)
`requestedBy`	UUID (FK → users)
`status`	Enum: PENDING_APPROVAL | APPROVED | RETURN_CONFIRMED | RETURN_REJECTED | CANCELLED
`returnReason`	String (nullable)
`reviewedBy`	UUID (nullable)
`reviewedAt`	Instant (nullable)
`providerReturnId`	String (nullable) — Xoxoday only, never XTRM
`providerConfirmedAt`	Instant (nullable)
`walletCreditedAt`	Instant (nullable)
`rejectionReason`	String (nullable)
`[inherited]`	id UUID, clientId UUID, createdAt Instant, updatedAt Instant

PART 2 — SERVICE + XTRM API MAPPING

Class: WalletService

Method: getWallet(UUID userId, CurrencyType) → WalletResponse

Returns the wallet for the user and currency type. Lazy-initialises the wallet on XTRM if it does not yet exist.

XTRM API: POST /API/V4/Wallet/CreateUserWallet (called only during lazy init)

```
Request:
{
  "CreateUserWallet": {
    "request": {
      "IssuerAccountNumber": "SPN Account Number",     // Required
      "UserID":              "PAT Account Number",     // Required - User.xtrmUserId
      "WalletName":          "tenXengage-CASH-{userId}", // Required - constructed
      "WalletCurrency":      "USD"                     // Required - CurrencyType.CASH -> ISO 4217
    }
  }
}

Response:
{
  "CreateUserWalletResponse": {
    "WalletID":   3201,         // Store as User.xtrmWalletId
    "WalletName": "tenXengage-CASH-...",
    "OperationStatus": { "Success": true, "Errors": null }
  }
}
```

XTRM Field	tenXengage Source	Status	Notes
`IssuerAccountNumber`	@Value(xtrm.issuer-account-number)	■ Config	Provided by XTRM at onboarding
`UserID`	User.xtrmUserId	■ **GAP**	Add xtrmUserId field to users table. Populated by CheckUserExist/CreateUser flow
`WalletName`	"tenXengage-CASH-" + userId	■ Derivable	Constructed — must be unique per user
`WalletCurrency`	CurrencyType.CASH → "USD"	■ **Mapping needed**	Add static map or cashIsoCurrencyCode to ClientRedemptionConfig

Method: getCompanyWallet(UUID partnerCompanyId, CurrencyType) → WalletResponse

Returns the company wallet for the partner company. Lazy-initialises XTRM company wallet if not yet created.

XTRM API: POST /API/V4/Wallet/CreateCompanyWallet (called only during lazy init)

```
Request:
{
  "CreateCompanyWallet": {
    "request": {
      "IssuerAccountNumber":      "SPN Account Number",             // Required
```

```
      "WalletName":               "tenXengage-COMPANY-CASH-{id}",   // Required
      "WalletCurrency":           "USD",                            // Required
      "WalletType":               "Standard",                       // Required - Standard or Accrual
      "AllowAccessAccountNumber": null                              // Optional
    }
  }
}

Response:
{
  "CreateCompanyWalletResponse": {
    "WalletID":       75420,    // Store as PartnerCompany.xtrmWalletId
    "WalletName":     "tenXengage-COMPANY-CASH-...",
    "WalletCurrency": "USD",
    "OperationStatus": { "Success": true, "Errors": null }
  }
}
```

XTRM Field	tenXengage Source	Status	Notes
`IssuerAccountNumber`	@Value(xtrm.issuer-account-number)	■ Config	Static config
`WalletName`	"tenXengage-COMPANY-CASH-" + partnerCompanyId	■ Derivable	Must be unique
`WalletCurrency`	CurrencyType.CASH → "USD"	■ **Mapping needed**	Same mapping as user wallet
`WalletType`	"Standard"	■ Hardcoded	Confirm with XTRM if "Accrual" needed
`AllowAccessAccountNumber`	null	■ Not required	Leave null for payout-only flow

■ *credit(), reserve(), debit(), release(), returnCredit() — these are INTERNAL wallet operations only. No XTRM API calls. debit() and release() are triggered BY XTRM webhook events, not callers of XTRM.*

Class: XtrmAdapter

Primary XTRM integration class. Orchestrates a 3-step flow for every payout: (1) Authenticate, (2) Check/Register Recipient, (3) Submit Transfer.

Step 1 — Authentication

XTRM API: POST /oAuth/token

```
Content-Type: application/x-www-form-urlencoded

Body: grant_type=password&client;_id={xtrm.client-id}&client;_secret={xtrm.client-secret}

Response:
{
  "access_token":  "eyJ...",   // Cache in-memory with expiry tracking
  "token_type":    "Bearer",
  "expires_in":    3600,
  "refresh_token": "..."        // Use for token refresh before expiry
}
```

XTRM Field	tenXengage Source	Notes
`client_id`	@Value(xtrm.client-id)	Never log — keep in secrets manager
`client_secret`	@Value(xtrm.client-secret)	Never log
`grant_type`	Hardcoded "password"	Static constant

Step 2 — Recipient Lookup

XTRM API: POST /API/V4/Beneficiary/CheckUserExist

```
Request:
{
  "CheckUserExist": {
    "request": {
      "IssuerAccountNumber": "SPN Account Number",   // Required
      "Email":               "recipient@email.com"   // Required - User.email
    }
  }
}
Response:
{
  "CheckUserExistResponse": {
    "CheckUserExistResult": {
      "UserId":     "PAT-XXXXX",   // null if user does not exist
      "UserExists": true,
      "OperationStatus": { "Success": true, "Errors": null }
    }
  }
}
```

XTRM Field	tenXengage Source	Status	Notes
`IssuerAccountNumber`	@Value(xtrm.issuer-account-number)	■ Config	Static
`Email`	User.email (resolved from tx.requestedBy UUID)	■ Available	Unique lookup key on XTRM side

■ *Decision: if UserExists=true → use returned UserId as RecipientUserId in TransferFund. If UserExists=false → call CreateUser (Step 2b). Cache UserId in User.xtrmUserId to skip this lookup on future redemptions.*

Step 2b — Recipient Registration (only if not found)

XTRM API: POST /API/V4/Register/CreateUser

```
Request:
{
  "CreateUser": {
    "request": {
      "IssuerAccountNumber": "SPN Account Number",   // Required
      "LegalFirstName":      "John",                 // Required - User.firstName
      "LegalLastName":       "Doe",                  // Required - User.lastName
      "EmailAddress":        "john@example.com",     // Required - User.email
      "EmailNotification":   "true",                 // Required
```

```
      "MobilePhone":         "+1-555-0100",          // Optional - User.phoneNumber
      "TaxId":               "XXX-XX-XXXX",          // Optional - User.taxId  !! GAP !!
      "DateOfBirth": {
        "Day":   "15",                                // Required - !! GAP !!
        "Month": "06",
        "Year":  "1985"
      },
      "Address": {
        "AddressLine1": "123 Main St",                // Required - User.address.line1
        "City":         "New York",                   // Required
        "Country":      "United States",              // Required - full country name
        "CountryISO2":  "US",                         // Required - ISO 3166-1 alpha-2
        "PostalCode":   "10001",                      // Required
        "Region":       "NY"                          // Optional
      }
    }
  }
}

Response:
{
  "CreateUserResponse": {
    "CreateUserResult": {
      "UserId": "PAT-XXXXX",   // Store as User.xtrmUserId
      "OperationStatus": { "Success": true, "Errors": null }
    }
  }
}
```

XTRM Field	tenXengage Source	Status	How to Obtain if Missing
`IssuerAccountNumber`	@Value(xtrm.issuer-account-number)	■ Available	Static config
`LegalFirstName`	User.firstName	■ Available	Direct mapping
`LegalLastName`	User.lastName	■ Available	Direct mapping
`EmailAddress`	User.email	■ Available	Direct mapping
`MobilePhone`	User.phoneNumber	■ **May be null**	Optional — send if present, skip if null
`TaxId`	User.taxId	■ **GAP**	Add encrypted taxId to UserProfile. Collect at onboarding or first CASH redemption via KYC prompt. Alternatively use XTRM's native tax form flow.
`DateOfBirth (Day/Month/Year)`	User.dateOfBirth	■ **GAP**	Add dateOfBirth (LocalDate) to UserProfile. Collect at onboarding.
`Address.AddressLine1`	User.address.line1	■ **May be incomplete**	Validate completeness at first CASH redemption. Prompt user if missing.
`Address.CountryISO2`	User.address.countryCode	■ **Needs validation**	Ensure ISO 3166-1 alpha-2 stored on user

■ *After successful CreateUser: store UserId in User.xtrmUserId (new field needed). On all future redemptions, skip Steps 2 and 2b — use cached xtrmUserId directly.*

Step 3 — Submit Cash Payout (Individual)

Called from: XtrmAdapter.submitPayout(RedemptionTransaction tx)

XTRM API: POST /API/V4/Fund/TransferFund

```
Request:
{
  "TransferFund": {
    "request": {
      "Transaction": {
        "IssuerAccountNumber": "SPN Account Number",    // Required
        "PaymentType":         "Personal",              // Required - Personal|Company
        "PaymentMethodId":     "XTR94503",              // Required !! GAP - see GetPaymentMethods !!
        "ProgramId":           "PROG-001",              // Required !! GAP - see GetPrograms !!
        "WalletID":            3170,                    // Required !! GAP - see GetCompanyWallets !!
        "PaymentDescription":  "Redemption: Cash Payout", // Required - constructed
        "PaymentCurrency":     "USD",                   // Required - CurrencyType.CASH -> ISO 4217
        "EmailNotification":   "true",                  // Required
        "TransactionDetails": [
          {
            "IssuerTransactionId":    "our-tx-uuid",    // Required - tx.id.toString()
            "PaymentAmount":          "50.00",          // Required - tx.amount (2dp string)
            "PartnerAccountNumber":   "SPN Account",    // Required - same as IssuerAccountNumber
            "RecipientUserId":        "PAT-XXXXX",      // Required - User.xtrmUserId
            "UserPrepaidVisaEmailID": "john@example.com", // Optional - User.email (Xtrm Choice)
            "UserLinkedBankID":       null,             // Optional - bank transfer
            "UserPayPalEmailID":      null,             // Optional - PayPal
            "Comment":               "tenXengage clientId=..."  // Optional
          }
        ]
      }
    }
  }
}

Response (Success):
{
  "TransferFundResponse": {
    "TransferFundResult": {
      "TransactionId": "XTRM-TXN-XXXXX",   // -> RedemptionTransaction.providerTransactionId
      "OperationStatus": { "Success": true, "Errors": null }
    }
  }
}

Response (Failure):
{
  "TransferFundResponse": {
    "TransferFundResult": {
      "TransactionId": null,
      "OperationStatus": { "Success": false, "Errors": "..." }  // -> failureReason
```

```
    }
  }
}
```

XTRM Field	tenXengage Source	Status	Notes
IssuerAccountNumber	@Value(xtrm.issuer-account-number)	■ Config	Provided by XTRM at onboarding
PaymentType	wallet.ownerType == INDIVIDUAL ? "Personal" : "Company"	■ Derivable	Simple enum mapping
PaymentMethodId	@Value(xtrm.payment-method-id)	■ **GAP**	Fetch via GetUserPaymentMethods. Use XTRM Choice (XTR94503) for recipient flexibility
ProgramId	@Value(xtrm.program-id)	■ **GAP**	Create Redemption program in XTRM portal. Fetch via GetPrograms. Store in config.
WalletID	@Value(xtrm.company-wallet-id)	■ **GAP**	Source wallet funding all payouts. Fetch via GetCompanyWallets after account is funded.
PaymentDescription	"Redemption: " + catalogItem.name	■ Derivable	Dynamic
PaymentCurrency	CurrencyType.CASH → "USD"	■ **Mapping needed**	Add cashIsoCurrencyCode to ClientRedemptionConfig (default USD)
IssuerTransactionId	tx.id.toString()	■ Available	CRITICAL — must be unique. Used for webhook reconciliation.
PaymentAmount	tx.amount.setScale(2,HALF_UP).toPlainString()	■ Available	BigDecimal → 2dp String
RecipientUserId	User.xtrmUserId	■ **GAP**	Requires xtrmUserId field on User entity (from Step 2/2b)
UserPrepaidVisaEmailID	User.email	■ Available	Enables Xtrm Choice — recipient picks bank/Visa/PayPal
Comment	"tenXengage clientId=" + tx.clientId	■ Derivable	Audit trail

■ *Post-call actions: Success=true → set tx.providerTransactionId = TransactionId, tx.status = PROCESSING. Success=false (4xx) → throw VendorRejectedException, tx.status = FAILED, call walletService.release(). 5xx/timeout → throw VendorUnavailableException (retriable via @Retryable, max 3 attempts).*

Step 3b — Submit Cash Payout (Company)

XTRM API: POST /API/V4/Fund/TransferFundtoCompany (when ownerType = COMPANY)

```
Request:
{
  "TransferFundtoCompany": {
    "request": {
      "IssuerAccountNumber":      "SPN Account Number",
      "PaymentType":              "Company",
      "PaymentMethodId":          "XTR94503",
      "ProgramId":                "PROG-001",
      "WalletID":                 3170,
      "Description":              "Redemption: Cash Payout",
      "Currency":                 "USD",
```

```
      "Amount":                  "50.00",
      "EmailNotification":       "true",
      "IssuerTransactionId":     "our-tx-uuid",          // tx.id.toString()
      "BeneficiaryAccountNumber": "SPN-PARTNER-ACCT",     // !! GAP PartnerCompany.xtrmAccountNumber !!
      "BeneficiaryWalletID":     75420,                  // !! GAP PartnerCompany.xtrmWalletId !!
      "BeneficiaryLinkedBankID":  null,
      "BeneficiaryPayPalEmailID": null
    }
  }
}
```

■ *Additional fields needed on PartnerCompany entity: xtrmAccountNumber (String) — XTRM SPN Account Number; xtrmWalletId (String) — returned from CreateCompanyWallet.*

Class: XtrmWebhookService

Endpoint: POST /api/v1/redemption/webhook/xtrm | Auth: permitAll (HMAC verified in handler)

Method: handleWebhook(String rawPayload, String hmacSignature) → void

Inbound payload from XTRM (expected structure):

```
{
  "externalId":           "our-tx-uuid",       // Maps to RedemptionTransaction.id (IssuerTransactionId)
  "status":               "SUCCEEDED",         // SUCCEEDED | FAILED | CANCELLED
  "providerTransactionId": "XTRM-TXN-XXXXX",   // XTRM internal reference
  "amount":               "50.00",
  "currency":             "USD",
  "timestamp":            "2026-04-22T10:30:00Z"
}
```

Status Mapping:

XTRM Status	tenXengage tx.status	WalletService Call	Additional Actions
SUCCEEDED	COMPLETED	walletService.debit(walletId, amount, currencyType, txId)	Set completedAt = now(). Publish domain event.
FAILED	FAILED	walletService.release(walletId, amount, currencyType, txId)	Set failureReason. Publish domain event.
CANCELLED	CANCELLED	walletService.release(walletId, amount, currencyType, txId)	Publish domain event.

Processing Steps (in strict order):

- 1. Verify HMAC-SHA256 using @Value(xtrm.webhook-secret) — throw WebhookAuthenticationException if invalid
- 2. Parse payload → XtrmWebhookEvent
- 3. Load RedemptionTransaction by externalId (= IssuerTransactionId sent in TransferFund)
- 4. Idempotency check — if status already COMPLETED/FAILED/CANCELLED, log and return silently
- 5. Call appropriate WalletService method based on XTRM status
- 6. Set tx.providerTransactionId and status. Save.
- 7. Publish domain event for notification service
- 8. Wrap entire method in try-catch — return HTTP 200 to XTRM even on internal errors (dead-letter log failures)

Class: RedemptionCatalogService

■ *syncXtrmCatalog(): XTRM has NO catalog/products API — it is a payout service, not a reward catalog. The syncCatalog() method in the document applies to Xoxoday only. CASH catalog items must be manually seeded into RedemptionCatalogItem with category=CASH, providerItemId=null, isReturnable=false. No XTRM API call needed.*

Class: ReturnsService

✕ *No XTRM API calls. Business rule in submitReturnRequest() explicitly throws BusinessRuleException if catalogItem.category == CASH. No XTRM return endpoint exists or is needed.*

PART 3 — FULL XTRM API CALL INVENTORY

#	XTRM Endpoint	Method	Called From	Trigger
1	/oAuth/token	POST	XtrmAdapter	App startup + token expiry
2	/API/V4/Beneficiary/CheckUserExist	POST	XtrmAdapter.submitPayout()	Every payout (check cache first)
3	/API/V4/Register/CreateUser	POST	XtrmAdapter.submitPayout()	Only when user not found in step 2
4	/API/V4/Wallet/CreateUserWallet	POST	WalletService.getWallet()	Lazy init of individual user wallet
5	/API/V4/Wallet/CreateCompanyWallet	POST	WalletService.getCompanyWallet()	Lazy init of company wallet
6	/API/V4/Fund/TransferFund	POST	XtrmAdapter.submitPayout()	INDIVIDUAL cash payout
7	/API/V4/Fund/TransferFundtoCompany	POST	XtrmAdapter.submitPayout()	COMPANY cash payout
8	/API/V4/Wallet/GetCompanyWallets	POST	Config init / admin tool	Get source WalletID at setup
9	/API/V4/Programs/GetPrograms	POST	Config init / admin tool	Get ProgramId at setup
10	/API/V4/Payment/GetPaymentMethods	POST	Config init / admin tool	Get PaymentMethodId at setup
–	(Inbound) Webhook	—	XtrmWebhookService	XTRM posts payout status updates

PART 4 — DATA GAPS

The following fields are required by XTRM but are not currently present in the tenXengage data model as described in the document.

#	Missing Field	Required By	Where to Add	How to Obtain
G1	User.xtrmUserId	CreateUserWallet, TransferFund.RecipientUserId	Add to users table	Returned from CheckUserExist or CreateUser. Cache permanently.
G2	User.xtrmWalletId	CreateUserWallet (reference storage)	Add to users table	Returned from CreateUserWallet. Cache permanently.
G3	User.taxId	CreateUser.TaxId	Add encrypted column to user_profiles	Collect at onboarding or first CASH redemption KYC. Consider XTRM native tax form flow.
G4	User.dateOfBirth	CreateUser.DateOfBirth	Add LocalDate to user_profiles	Collect at onboarding
G5	User.address (full)	CreateUser.Address	Validate completeness on users table	Prompt user to complete at first CASH redemption
G6	PartnerCompany.xtrmAccountNumber	TransferFundtoCompany.BeneficiaryAccountNumber	Add to partner_companies table	Obtain from XTRM when company is registered as beneficiary
G7	PartnerCompany.xtrmWalletId	TransferFundtoCompany.BeneficiaryWalletID	Add to partner_companies table	Returned from CreateCompanyWallet
G8	xtrm.program-id (config)	TransferFund.ProgramId	application.properties / secrets	Create Redemption program in XTRM portal. Fetch via GetPrograms.
G9	xtrm.company-wallet-id (config)	TransferFund.WalletID	application.properties / secrets	Fetch once via GetCompanyWallets after XTRM account is funded
G10	xtrm.payment-method-id (config)	TransferFund.PaymentMethodId	application.properties / secrets	Fetch once via GetUserPaymentMethods. Use XTRM Choice for flexibility.
G11	xtrm.webhook-secret (config)	XtrmWebhookService HMAC verification	Secrets manager	Provided by XTRM when registering webhook endpoint
G12	ISO currency mapping	All wallet and payout APIs	Static map in XtrmAdapter or ClientRedemptionConfig.cashIsoCurrencyCode	Add cashIsoCurrencyCode (default USD) to ClientRedemptionConfig

PART 5 — OPEN QUESTIONS & AMBIGUITIES

OQ-1 Wrong Endpoint in Document ■ HIGH

The document specifies POST /v1/rewards as the XTRM payout endpoint. The actual XTRM XAPI uses POST /API/V4/Fund/TransferFund with a completely different request schema. Code generated from the document will not work against real XTRM.

Action: *Confirm with XTRM account team before XtrmAdapter is coded.*

OQ-2 apidoc.xtrm.com Is Inaccessible ■ HIGH

The URL provided (apidoc.xtrm.com/#9b57822a-...) renders as a JavaScript SPA that cannot be crawled. The referenced section may contain different schemas than the legacy XAPI console used in this analysis.

Action: *Access apidoc.xtrm.com with an authenticated browser session and compare endpoint schemas.*

OQ-3 ProgramId Purpose and Selection ■ MEDIUM

XTRM Programs categorise transactions (SPIFF, MDF, rebate, etc). ProgramId is required for TransferFund. It is unclear which program type maps to a redemption payout.

Action: *Create a dedicated Redemption Store program in the XTRM portal. Confirm TransactionCategoryID with XTRM.*

OQ-4 PaymentMethodId and Xtrm Choice ■ MEDIUM

XTRM supports multiple payout methods (bank transfer, prepaid Visa, PayPal). The document assumes email-linked Visa. Should tenXengage use Xtrm Choice (recipient picks) or direct-to-bank?

Action: *Confirm payout method strategy with product owner. Xtrm Choice is lowest friction — populate UserPrepaidVisaEmailID with recipient email.*

OQ-5 XTRM Webhook Payload Schema ■ HIGH

XTRM's exact webhook event field names are not publicly documented. The document assumes externalId, status, providerTransactionId. Actual field names need verification.

Action: *Register webhook in XTRM sandbox, trigger a test payout, and capture the real payload before coding XtrmWebhookService.*

OQ-6 Company Payout Flow ■ MEDIUM

TransferFundtoCompany requires BeneficiaryAccountNumber (the partner company's XTRM SPN). It is unclear if partner companies are registered on XTRM as issuers or just as beneficiary users.

Action: *Clarify with XTRM. If company accounts are not supported, route company payouts through the representative user's individual account.*

OQ-7 TaxId Collection and Compliance ■ HIGH

TaxId (SSN/EIN) is sensitive PII subject to regulatory controls. Collecting it in tenXengage introduces 1099-K compliance burden. XTRM can alternatively collect tax forms natively.

Action: *Evaluate XTRM's native tax form flow. If usable, make CreateUser.TaxId optional and avoid storing SSN in tenXengage.*

OQ-8 Idempotency on TransferFund ■ HIGH

If XtrmAdapter.submitPayout() is retried due to VendorUnavailableException, will XTRM process the same IssuerTransactionId a second time (double-payment)?

Action: *Confirm XTRM idempotency guarantee on IssuerTransactionId. If not guaranteed, add a pre-retry guard checking whether the transaction is already PROCESSING.*

OQ-9 XTRM Wallet Balance Monitoring ■ MEDIUM

The XTRM source wallet must have sufficient funds. There is no mechanism to monitor or auto-replenish it. If the wallet runs dry, all redemptions will fail.

Action: *Add a scheduled job calling GetCompanyWallets to check balance. Alert when below a threshold.*

OQ-10 BATCH Processing Mode ■ MEDIUM

BATCH mode sets tx.status=PENDING and schedules the payout for the next batch run. The batch runner is not defined anywhere in the document.

Action: *Define a @Scheduled batch runner in RedemptionOrchestrationService querying PENDING transactions and calling routeToVendor().*

OQ-11 Duplicate Email Notifications ■ LOW

When EmailNotification=true in TransferFund, XTRM sends its own email. The tenXengage notification service may send a second one via the webhook domain event.

Action: *Set EmailNotification=false and rely on tenXengage notifications, or inform the user they will receive two emails.*

OQ-12 Multi-Currency Support ■ MEDIUM

CurrencyType.CASH is currently mapped to USD. For partners in other regions (India, UK etc.), this mapping breaks.

Action: *Add cashIsoCurrencyCode (String, ISO 4217, default USD) to ClientRedemptionConfig.*

PART 6 — REST CONTROLLERS (XTRM-RELEVANT ENDPOINTS)

Controller	Endpoint	HTTP	XTRM Involvement
`RedemptionController`	/api/v1/redemption/requests	POST	→ OrchestrationService → XtrmAdapter.submitPayout() if CASH + INSTANT
`RedemptionController`	/api/v1/redemption/requests/company	POST	→ Same flow using TransferFundtoCompany
`RedemptionController`	/api/v1/redemption/requests/{id}/approve	POST	→ OrchestrationService.approveRedemption() → XtrmAdapter.submitPayout()
`RedemptionWebhookController`	/api/v1/redemption/webhook/xtrm	POST	← Inbound from XTRM, handled by XtrmWebhookService (HMAC secured)
`WalletController`	/api/v1/wallets/me	GET	No XTRM call — returns internal wallet state only
`WalletController`	/api/v1/wallets/company/{companyId}	GET	No XTRM call — returns internal wallet state only